

시스템 프로그래밍 Project #4

Mallocator 보고서

학번: 20221531

이름: 김리나

1. 개발 목표

메모리 할당과 관련된 memory allocator 함수 malloc, free, realloc 함수들을 직접 구현한다.

2. 개발 범위 및 설명

```
int mm_init(void);
void *mm_malloc(size_t size);
void mm_free(void *ptr);
void *mm_realloc(void *ptr, size_t size);
```

(1) int mm_init(void)

할당기 초기화를 담당하는 함수로, 힙의 시작 부분에 프로로그 블록과 에필로그 블록을 설정하고, 초깃값을 가진 free block을 생성하여 가용 리스트를 준비한다.

(2) void *mm_malloc(size_t size)

사용자 요청 크기만큼 정렬된 블록을 할당하며, 내부 단편화를 고려해 최소 블록 크기 이상으로 요청 크기를 조정하고, free list에서 최적의 위치를 탐색해 할당을 수행한다.

(3) void mm_free(void *ptr)

사용자가 반환한 포인터를 기준으로 해당 블록의 헤더/푸터를 0으로 표시하여 미할당 상태로 전환하고, 인접한 free block들과 coalesce()를 통해 병합한다.

(4) void *mm_realloc(void *ptr, size_t size)

이미 할당되어 있는 블록의 크기를 사용자의 요청에 맞게 조절하며, 가능한 경우 기존 블록을 in-place로 확장하고, 그렇지 않으면 새 블록을 할당한 후 데이터를 복사한다.

3. 개발 내용

개발을 위해 추가한 매크로 상수와 매크로 함수, 전역 변수, 전역 함수들이다.

(1) 매크로 상수

```
#define WSIZE 4 //한 word 크기
#define DSIZE 8 // double word 정렬을 위한 사이즈, 헤더+푸터 = 8
#define CHUNKSIZE (1 << 12) // 기본 단위 메모리 크기,  $2^{12} = 4096$ bytes
```

(2) 매크로 함수

```
#define MAX(x, y) ((x) > (y) ? (x) : (y)) // 더 큰 값 반환
#define PACK(size, alloc) ((size) | (alloc)) // 크기와 할당 여부를 하나의 unsigned int 값으로 결합
#define GET(p) (*(unsigned int *)(p)) // p가 가리키는 메모리 위치에서 4바이트 값 읽어오기
#define PUT(p, val) (*(unsigned int *)(p) = (val)) // p가 가리키는 메모리 위치에서 4바이트 val 값 저장
#define GET_SIZE(p) (GET(p) & ~0x7) // 블록 크기 추출
#define GET_ALLOC(p) (GET(p) & 0x1) // 할당 비트 추출 1이면 할당됨, 0이면 가용
#define HDRP(bp) ((char *)(bp) - WSIZE) // 블록 포인터로부터 헤더 주소 계산
#define FTRP(bp) ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE) // 블록 포인터로부터 푸터 주소 계산
#define NEXT_BLKP(bp) ((char *)(bp) + GET_SIZE(((char *)(bp) - WSIZE))) // 다음 블록의 payload 포인터 계산
#define PREV_BLKP(bp) ((char *)(bp) - GET_SIZE(((char *)(bp) - DSIZE))) // 이전 블록의 payload 포인터 계산
#define NEXT_FREE(bp) (*(char **)(bp)) // 가용 블록에서 다음 free block 포인터 읽기
#define PREV_FREE(bp) (*(char **)(bp) + 1) // 가용 블록에서 이전 free block 포인터 읽기
```

(3) 전역 변수

```
/* heap_listp: 힙의 시작 위치를 가리키는 포인터  
   프로로그 블록 이후 첫 유효 블록부터 힙 전체를 순회할 때 기준이  
   됨*/  
static char *heap_listp = 0;  
  
/* free_listp: 명시적 가용 리스트의 첫 번째 free 블록을 가리키는  
   포인터  
   명시적 가용 리스트는 이 포인터를 시작으로 연결된 free 블록들을  
   탐색함*/  
static char *free_listp = 0;
```

(4) 전역 함수

```
static void *extend_heap(size_t words); // 힙을 주어진 크기만큼  
확장  
static void *coalesce(void *bp); // 인접한 가용 블록들과 병합  
static void insert_free_block(void *bp); // 가용 블록을 free  
list 에 삽입  
static void remove_free_block(void *bp); // 가용 블록을 free  
list 에서 제거  
static void *find_fit(size_t asize); // 적절한 크기의 가용 블록  
탐색  
static void place(void *bp, size_t asize); // 블록을 할당하고  
필요한 경우 분할
```

4. 함수 설명

구현한 `mm_init`, `mm_malloc`, `mm_free`, `mm_realloc` 함수와 이를 구현하기 위해 작성한 함수들의 설명이다.

(1) `int mm_init(void)`

이 함수는 할당기의 초기화를 담당한다.

초기 힙 공간을 `mem_sbrk()`을 통해 할당한 후, 정렬 패딩과 함께 프로로그(prologue) 블록과 에필로그(epilogue) 블록을 구성한다. 또한 첫 가용 블록을 생성하기 위해 `extend_heap()`을 호출하며, 가용 리스트의 시작 포인터인 `free_listp`도 초기화된다.

(2) `static void *extend_heap(size_t words)`

사용 가능한 블록이 부족할 경우, 힙을 지정된 워드 수만큼 확장하여 새로운 free block을 생성한다. 확장된 블록은 초기화된 후 `coalesce()`를 통해 인접한 free block과 병합된다.

(3) `static void *coalesce(void *bp)`

인접한 이전 또는 다음 블록이 가용 상태인 경우, 해당 블록들과 병합하여 더 큰 가용 블록으로 만들어 단편화를 줄인다. 병합된 블록은 free list에 삽입된다.

(4) `static void insert_free_block(void *bp)`

입력된 free block을 명시적 가용 리스트의 맨 앞에 삽입한다. 블록은 이중 연결 리스트를 통해 관리되며, `NEXT_FREE`와 `PREV_FREE` 포인터를 설정한다.

(5) `static void remove_free_block(void *bp)`

할당 등으로 인해 더 이상 가용 블록이 아닌 경우, 해당 블록을 free list에서 제거한다. 이 역시 이중 연결 리스트의 포인터를 이용하여 수행된다.

(6) `void *mm_malloc(size_t size)`

사용자로부터 요청받은 크기의 메모리를 할당하는 함수이다. 내부 단편화를 고려해 `ALIGN()`을 통해 크기를 정렬하고, `find_fit()`으로 적절한 free block을 탐색한 후, `place()`를 통해 블록을 할당한다. 적절한 free block이 없을 경우 `extend_heap()`을 통해 힙을 확장한다.

(7) static void *find_fit(size_t asize)

best-fit 전략에 따라 가용 리스트에서 요청 크기 이상이면서 가장 작은 블록을 찾아 반환한다. 완벽히 일치하는 크기가 있다면 바로 반환하여 탐색 비용을 줄인다.

(8) static void place(void *bp, size_t asize)

지정된 블록에 메모리를 실제로 할당하며, 남는 공간이 충분할 경우 이를 분할하여 새로운 free block으로 만들고 가용 리스트에 삽입한다.

(9) void mm_free(void *bp)

사용이 끝난 블록을 해제하는 함수이다. 해당 블록의 헤더와 푸터를 수정하여 가용 상태로 표시하고, coalesce()를 호출하여 인접 블록과 병합한 후 free list에 삽입한다.

mm-naive.c에선 함수 인자명이 *ptr이었는데 내부 매크로(HDRP, FTRP 등)와의 일관성과 가독성을 높이기 위해 *bp로 변경했다.

(10) void *mm_realloc(void *ptr, size_t size)

이미 할당된 블록의 크기를 변경하는 함수로, ptr == NULL이면 malloc으로, size == 0이면 free로 처리한다. 블록을 in-place로 확장 가능한 경우 병합하여 확장하고, 그렇지 않으면 새로운 블록을 할당한 뒤 데이터를 복사하고 기존 블록을 해제한다.

5. 성능 평가 결과

제공받은 mdriver를 -V 옵션으로 실행하여 구현한 mallocato의 성능 평가를 확인하였다.

```
cse20221531@cspro:~/prj4-malloc$ ./mdriver -V
[20221531]::NAME: Lina Kim, Email Address: kimrn142@gmail.com
Using default tracefiles in ./tracefiles/
Measuring performance with gettimeofday().

Testing mm malloc
Reading tracefile: amptjp-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: cccp-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: cp-decl-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: expr-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: coalescing-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: random-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: random2-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: binary-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: binary2-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: realloc-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.
Reading tracefile: realloc2-bal.rep
Checking mm_malloc for correctness, efficiency, and performance.

Results for mm malloc:
trace  valid  util    ops    secs  Kops
0      yes   99%   5694  0.000424 13432
1      yes   99%   5848  0.000446 13127
2      yes   99%   6648  0.000468 14211
3      yes  100%   5380  0.000446 12076
4      yes   66%  14400  0.000374 38503
5      yes   96%   4800  0.004391  1093
6      yes   95%   4800  0.004638  1035
7      yes   55%  12000  0.032751   366
8      yes   51%  24000  0.103240   232
9      yes   87%  14401  0.000200 72077
10     yes   67%  14401  0.000203 70976
Total          83% 112372  0.147579   761

Perf index = 50 (util) + 40 (thru) = 90/100
```

mdriver의 결과에 따르면, 구현한 메모리 할당기는 모든 trace에서 정상 동작 (valid = yes)했으며, 평균 공간 활용도 83%, 평균 처리 속도 761Kops를 기록하여 총 성능 점수 90점(50(util) + 40(thru))이 나왔다. 특히 초기 trace들에서는 99~100%의 높은 활용도를 보였고, 일부 큰 block 위주의 trace에서는 상대적으로 낮은 성능을 보이는 것을 확인할 수 있다.